# Modules over Ore rings

***Release 10.10***

**The Sage Development Team**

**Oct 28, 2026**

# CONTENTS


1 Modules, submodules and quotients 3

2 Morphisms 59

Python Module Index 81

Index 83

Let $R$ be a commutative ring, $\theta : K \to K$ by a ring endomorphism and $\partial : K \to K$ be a $\theta$-derivation, that is an additive map satisfying the following axiom

$$\partial(xy) = \theta(x)\partial(y) + \partial(x)y$$

The Ore polynomial ring associated to these data is $\mathcal{S} = R[X; \theta, \partial]$; its elements are the usual polynomials over $R$ but the multiplication is twisted according to the rule

$$\partial(xy) = \theta(x)\partial(y) + \partial(x)y$$

We refer to `OrePolynomialRing` for more details.

A Ore module over $(R, \theta, \partial)$ is by definition a module over $\mathcal{S}$; it is the same than a $R$-module $M$ equipped with an additive $f : M \to M$ such that

$$f(ax) = \theta(a)f(x) + \partial(a)x$$

Such a map $f$ is called a pseudomorphism (see also `sage.modules.free_module.FreeModule_generic.pseudohom()`).

SageMath provides support for creating and manipulating Ore modules that are finite free over the base ring $R$. This includes, in particular, Frobenius modules and modules with connexions.

CHAPTER

ONE

# MODULES, SUBMODULES AND QUOTIENTS

## 1.1 Ore modules

Let $R$ be a commutative ring, $\theta : K \to K$ by a ring endomorphism and $\partial : K \to K$ be a $\theta$-derivation, that is an additive map satisfying the following axiom

$$\partial(xy) = \theta(x)\partial(y) + \partial(x)y$$

A Ore module over $(R, \theta, \partial)$ is a $R$-module $M$ equipped with a additive $f : M \to M$ such that

$$f(ax) = \theta(a)f(x) + \partial(a)x$$

Such a map $f$ is called a pseudomorphism.

Equivalently, a Ore module is a module over the (noncommutative) Ore polynomial ring $\mathcal{S} = R[X; \theta, \partial]$.

### Defining Ore modules

SageMath provides support for creating and manipulating Ore modules that are finite free over the base ring $R$.

To start with, the method `sage.rings.polynomial.ore_polynomial_ring.OrePolynomialRing.quotient_module()` creates the quotient $\mathcal{S}/\mathcal{S}P$, endowed with its structure of $\mathcal{S}$-module, that is its structure of Ore module:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,) = S._
↪first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
>>> M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

Classical methods are available and we can work with elements in $M$ as we do usually for vectors in finite free modules:

```
sage: M.basis()
[(1, 0), (0, 1)]
```

(continues on next page)

(continued from previous page)

```
sage: v = M((z, z^2)); v
(z, z^2)
sage: z*v
(z^2, 2*z + 2)
```

```
>>> from sage.all import *
>>> M.basis()
[(1, 0), (0, 1)]

>>> v = M((z, z**Integer(2))); v
(z, z^2)
>>> z*v
(z^2, 2*z + 2)
```

The Ore action (or equivalently the structure of $\mathcal{S}$-module) is also easily accessible:

```
sage: X*v
(3*z^2 + 2*z, 2*z^2 + 4*z + 4)
```

```
>>> from sage.all import *
>>> X*v
(3*z^2 + 2*z, 2*z^2 + 4*z + 4)
```

The method *sage.modules.ore_module.OreModule.pseudohom()* returns the map $f$ defining the action of $X$:

```
sage: M.pseudohom()
Free module pseudomorphism (twisted by z |--> z^5) defined by the matrix
[  0   1]
[4*z   0]
Domain: Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
Codomain: Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^
↪5
```

```
>>> from sage.all import *
>>> M.pseudohom()
Free module pseudomorphism (twisted by z |--> z^5) defined by the matrix
[  0   1]
[4*z   0]
Domain: Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
Codomain: Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^
↪5
```

A useful feature is the possibility to give chosen names to the vectors of the canonical basis. This is easily done as follows:

```
sage: N.<u,v,w> = S.quotient_module(X^3 + z*X + 1)
sage: N
Ore module <u, v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: N.basis()
[u, v, w]
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(3) + z*X + Integer(1), names=('u', 'v', 'w',));␣
```

(continues on next page)

(continued from previous page)

```
↪(u, v, w,) = N._first_ngens(3)
>>> N
Ore module <u, v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> N.basis()
[u, v, w]
```

Alternatively, one can pass in the argument `names`; this could be useful in particular when we want to name the vectors basis $e_0, e_1, \ldots$:

```
sage: A = S.quotient_module(X^11 + z, names='e')
sage: A
Ore module <e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10> over Finite Field in z of␣
↪size 5^3 twisted by z |--> z^5
sage: A.basis()
[e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10]
```

```
>>> from sage.all import *
>>> A = S.quotient_module(X**Integer(11) + z, names='e')
>>> A
Ore module <e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10> over Finite Field in z of␣
↪size 5^3 twisted by z |--> z^5
>>> A.basis()
[e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10]
```

Do not forget to use the method `inject_variables()` to get the $e_i$ in your namespace:

```
sage: e0
Traceback (most recent call last):
...
NameError: name 'e0' is not defined
sage: A.inject_variables()
Defining e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10
sage: e0
e0
```

```
>>> from sage.all import *
>>> e0
Traceback (most recent call last):
...
NameError: name 'e0' is not defined
>>> A.inject_variables()
Defining e0, e1, e2, e3, e4, e5, e6, e7, e8, e9, e10
>>> e0
e0
```

### Submodules and quotients

SageMath provides facilities for creating submodules and quotient modules of Ore modules. First of all, we define the Ore module $\mathcal{S}/\mathcal{S}P^2$ (for some Ore polynomials $P$), which is obviously not simple:

```
sage: P = X^2 + z*X + 1
sage: U = S.quotient_module(P^2, names='u')
```

(continues on next page)

(continued from previous page)

```
sage: U.inject_variables()
Defining u0, u1, u2, u3
```

```
>>> from sage.all import *
>>> P = X**Integer(2) + z*X + Integer(1)
>>> U = S.quotient_module(P**Integer(2), names='u')
>>> U.inject_variables()
Defining u0, u1, u2, u3
```

We now build the submodule $\mathcal{S}P/\mathcal{S}P^2$ using the method *sage.modules.ore_module.OreModule.span()*:

```
sage: V = U.span(P*u0)
sage: V
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: V.basis()
[u0 + (z^2+2*z+2)*u2 + 4*z*u3,
 u1 + (2*z^2+4*z+4)*u2 + u3]
```

```
>>> from sage.all import *
>>> V = U.span(P*u0)
>>> V
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> V.basis()
[u0 + (z^2+2*z+2)*u2 + 4*z*u3,
 u1 + (2*z^2+4*z+4)*u2 + u3]
```

We underline that the span is really the $\mathcal{S}$-span and not the $R$-span (as otherwise, it will not be a Ore module).

As before, one can use the attributes `names` to give explicit names to the basis vectors:

```
sage: V = U.span(P*u0, names='v')
sage: V
Ore module <v0, v1> over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: V.inject_variables()
Defining v0, v1
sage: v0
v0
sage: U(v0)
u0 + (z^2+2*z+2)*u2 + 4*z*u3
```

```
>>> from sage.all import *
>>> V = U.span(P*u0, names='v')
>>> V
Ore module <v0, v1> over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> V.inject_variables()
Defining v0, v1
>>> v0
v0
>>> U(v0)
u0 + (z^2+2*z+2)*u2 + 4*z*u3
```

A coercion map from $V$ to $U$ is automatically created. Hence, we can safely combine vectors in $V$ and vectors in $U$ in a single expression:

```
sage: v0 - u0
(z^2+2*z+2)*u2 + 4*z*u3
```

```
>>> from sage.all import *
>>> v0 - u0
(z^2+2*z+2)*u2 + 4*z*u3
```

We can create the quotient $U/V$ using a similar syntax:

```
sage: W = U.quo(P*u0)
sage: W
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: W.basis()
[u2, u3]
```

```
>>> from sage.all import *
>>> W = U.quo(P*u0)
>>> W
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> W.basis()
[u2, u3]
```

We see that SageMath reuses by default the names of the representatives to denote the vectors in the quotient $U/V$. This behaviour can be overridden by providing explicit names using the attributes `names`.

Shortcuts for creating quotients are also available:

```
sage: U / (P*u0)
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: U/V
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> U / (P*u0)
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> U/V
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

### Morphisms of Ore modules

For a tutorial on morphisms of Ore modules, we refer to *sage.modules.ore_module_morphism*.

AUTHOR:

- Xavier Caruso (2024-10)
- Xavier Caruso (2025-08); add support for Ore modules over PIDs

**class** sage.modules.ore_module.**OreAction**

Bases: Action

Action by left multiplication of Ore polynomial rings over Ore modules.

**class** sage.modules.ore_module.**OreModule**(*mat*, *ore*, *denominator*, *names*, *category*)

Bases: UniqueRepresentation, FreeModule_ambient

Generic class for Ore modules.

**Element**

alias of *OreModuleElement*

**ambient_modules**()

Return the list of modules in which this module naturally lives.

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^2 + a
sage: M = S.quotient_module(P^3, names='e')
sage: M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(2) + a
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

For an ambient module, the list is reduced to one element (namely the module itself):

```
sage: M.ambient_modules()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> M.ambient_modules()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

On the contrary, for a submodule of $M$, the list also contains the ambient space:

```
sage: MP = M.span(P*e0)
sage: MP.ambient_modules()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = M.span(P*e0)
```

(continues on next page)

(continued from previous page)

```
>>> MP.ambient_modules()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

If we now create a submodule of $MP$, the list gets even longer:

```
sage: MP2 = MP.span(P^2*e0)
sage: MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = MP.span(P**Integer(2)*e0)
>>> MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We underline nevertheless that if we define $MP^2$ has a submodule of $M$, the intermediate $MP$ does not show up in the list:

```
sage: MP2 = M.span(P^2*e0)
sage: MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = M.span(P**Integer(2)*e0)
>>> MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

**basis**()

Return the canonical basis of this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
```

(continues on next page)

(continued from previous page)

```
sage: M = S.quotient_module(X^3 - z)
sage: M.basis()
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) - z)
>>> M.basis()
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
```

**covers**()

Return the list of modules of which this module is a quotient.

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^2 + a
sage: M = S.quotient_module(P^3, names='e')
sage: M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(2) + a
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

For an ambient module, the list is reduced to one element (namely the module itself):

```
sage: M.covers()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> M.covers()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We now create a quotient of $M$ and observe what happens:

```
sage: MP2 = M.quo(P^2*e0)
sage: MP2.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = M.quo(P**Integer(2)*e0)
>>> MP2.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

If we now create a quotient of $M/MP$, another item is added to the list:

```
sage: MP = MP2.quo(P*e0)
sage: MP.covers()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = MP2.quo(P*e0)
>>> MP.covers()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We underline nevertheless that if we directly define $M/MP$ has a quotient of $M$, the intermediate $M/MP^2$ does not show up in the list:

```
sage: MP = M.quo(P^2*e0)
sage: MP.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = M.quo(P**Integer(2)*e0)
>>> MP.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

**fitting_index**(*other=None*)

Return the generator of the Fitting ideal of the quotient of `other` by this module.

INPUT:

- `other` (default: `None`) – an Ore module; if `None`, the ambient space of this module

EXAMPLES:

```
sage: A.<t> = GF(3)[]
sage: f = A.hom([t+1])
sage: S.<X> = OrePolynomialRing(A, f)
sage: P = X^2 + t
sage: M = S.quotient_module(P^2, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3
```

```
>>> from sage.all import *
>>> A = GF(Integer(3))['t']; (t,) = A._first_ngens(1)
>>> f = A.hom([t+Integer(1)])
>>> S = OrePolynomialRing(A, f, names=('X',)); (X,) = S._first_ngens(1)
>>> P = X**Integer(2) + t
>>> M = S.quotient_module(P**Integer(2), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3
```

We create a submodule and compute its Fitting index:

```
sage: N = M.span(X^3*e0)
sage: N.fitting_index()
t^6 + t^4 + t^2
```

```
>>> from sage.all import *
>>> N = M.span(X**Integer(3)*e0)
>>> N.fitting_index()
t^6 + t^4 + t^2
```

Here is another example where the submodule has smaller rank; in this case, the Fitting index is $0$:

```
sage: MP = M.span(P*e0)
sage: MP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 3 twisted by t |--> t + 1
sage: MP.fitting_index()
0
```

```
>>> from sage.all import *
>>> MP = M.span(P*e0)
>>> MP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 3 twisted by t |--> t + 1
>>> MP.fitting_index()
0
```

Another example with two submodules of $M$:

```
sage: NP = M.span(X^3*P*e0)
sage: NP.fitting_index()  # index in M
0
sage: NP.fitting_index(MP)
t^3 + 2*t
```

```
>>> from sage.all import *
>>> NP = M.span(X**Integer(3)*P*e0)
>>> NP.fitting_index()  # index in M
0
>>> NP.fitting_index(MP)
t^3 + 2*t
```

We note that it is actually not necessary that `other` contains `self`; if it is not the case, a fraction is returned:

```
sage: MP.fitting_index(NP)
1/(t^3 + 2*t)
```

```
>>> from sage.all import *
>>> MP.fitting_index(NP)
1/(t^3 + 2*t)
```

**gen**(*i*)

Return the $i$-th vector of the canonical basis of this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^3 - z)
sage: M.gen(0)
(1, 0, 0)
sage: M.gen(1)
(0, 1, 0)
sage: M.gen(2)
(0, 0, 1)
sage: M.gen(3)
Traceback (most recent call last):
...
IndexError: generator is not defined
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) - z)
>>> M.gen(Integer(0))
(1, 0, 0)
>>> M.gen(Integer(1))
(0, 1, 0)
>>> M.gen(Integer(2))
(0, 0, 1)
```

(continues on next page)

(continued from previous page)

```
>>> M.gen(Integer(3))
Traceback (most recent call last):
...
IndexError: generator is not defined
```

**gens**()

Return the canonical basis of this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^3 - z)
sage: M.gens()
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) - z)
>>> M.gens()
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
```

**hom**(*im_gens*, *codomain=None*)

Return the morphism from this Ore module to `codomain` defined by `im_gens`.

INPUT:

- `im_gens` – a datum defining the morphism to build; it could either a list, a tuple, a dictionary or a morphism of Ore modules
- `codomain` (default: `None`) – a Ore module, the codomain of the morphism; if `None`, it is inferred from `im_gens`

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: P = X^3 + 2*t*X^2 + (t^2 + 2)*X + t
sage: Q = t*X^2 - X + 1

sage: U = S.quotient_module(P, names='u')
sage: U.inject_variables()
Defining u0, u1, u2
sage: V = S.quotient_module(P*Q, names='v')
sage: V.inject_variables()
Defining v0, v1, v2, v3, v4
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> P = X**Integer(3) + Integer(2)*t*X**Integer(2) + (t**Integer(2) +␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2))*X + t
>>> Q = t*X**Integer(2) - X + Integer(1)

>>> U = S.quotient_module(P, names='u')
>>> U.inject_variables()
Defining u0, u1, u2
>>> V = S.quotient_module(P*Q, names='v')
>>> V.inject_variables()
Defining v0, v1, v2, v3, v4
```

The first method for creating a morphism from $U$ to $V$ is to explicitly write down its matrix in the canonical bases:

```
sage: mat = matrix(3, 5, [1, 4, t, 0, 0,
....:                     0, 1, 0, t, 0,
....:                     0, 0, 1, 1, t])
sage: f = U.hom(mat, codomain=V)
sage: f
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> mat = matrix(Integer(3), Integer(5), [Integer(1), Integer(4), t,␣
↪Integer(0), Integer(0),
...                     Integer(0), Integer(1), Integer(0), t, Integer(0),
...                     Integer(0), Integer(0), Integer(1), Integer(1), t])
>>> f = U.hom(mat, codomain=V)
>>> f
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
```

This method is however not really convenient because it requires to compute beforehand all the entries of the defining matrix. Instead, we can pass the list of images of the generators:

```
sage: g = U.hom([Q*v0, X*Q*v0, X^2*Q*v0])
sage: g
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
sage: g.matrix()
[1 4 t 0 0]
[0 1 0 t 0]
[0 0 1 1 t]
```

```
>>> from sage.all import *
>>> g = U.hom([Q*v0, X*Q*v0, X**Integer(2)*Q*v0])
>>> g
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
>>> g.matrix()
[1 4 t 0 0]
[0 1 0 t 0]
[0 0 1 1 t]
```

One can even give the values of the morphism on a smaller set as soon as the latter generates the domain as Ore module. The syntax uses dictionaries as follows:

```
sage: h = U.hom({u0: Q*v0})
sage: h
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
sage: g == h
True
```

```
>>> from sage.all import *
>>> h = U.hom({u0: Q*v0})
>>> h
Ore module morphism:
  From: Ore module <u0, u1, u2> over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
>>> g == h
True
```

Finally `im_gens` can also be itself a Ore morphism, in which case SageMath tries to cast it into a morphism with the requested domains and codomains. As an example below, we restrict $g$ to a submodule:

```
sage: C.<c0,c1> = U.span((X + t)*u0)
sage: gC = C.hom(g)
sage: gC
Ore module morphism:
  From: Ore module <c0, c1> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt

sage: g(c0) == gC(c0)
True
sage: g(c1) == gC(c1)
True
```

```
>>> from sage.all import *
>>> C = U.span((X + t)*u0, names=('c0', 'c1',)); (c0, c1,) = C._first_ngens(2)
>>> gC = C.hom(g)
>>> gC
Ore module morphism:
  From: Ore module <c0, c1> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v0, v1, v2, v3, v4> over Fraction Field of Univariate␣
↪Polynomial Ring in t over Finite Field of size 5 twisted by d/dt

>>> g(c0) == gC(c0)
True
>>> g(c1) == gC(c1)
True
```

**identity_morphism**()

Return the identity morphism of this Ore module.

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<u,v> = S.quotient_module(X^2 + a*X + a^2)
sage: id = M.identity_morphism()
sage: id
Ore module endomorphism of Ore module <u, v> over Finite Field in a of size 7^
↪5 twisted by a |--> a^7

sage: id(u)
u
sage: id(v)
v
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + a*X + a**Integer(2), names=('u', 'v
↪',)); (u, v,) = M._first_ngens(2)
>>> id = M.identity_morphism()
>>> id
Ore module endomorphism of Ore module <u, v> over Finite Field in a of size 7^
↪5 twisted by a |--> a^7

>>> id(u)
u
>>> id(v)
v
```

**is_submodule**(*other*)

Return `True` if `other` is included in this module; `False` otherwise.

EXAMPLES:

```
sage: A.<t> = GF(3)[]
sage: f = A.hom([t+1])
sage: S.<X> = OrePolynomialRing(A, f)
sage: P = X^2 + t
sage: M = S.quotient_module(P^3, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
sage: MP = M.span(P*e0)
sage: MP2 = MP.span(P^2*e0)

sage: MP2.is_submodule(MP)
True
sage: MP.is_submodule(MP2)
False
```

```
>>> from sage.all import *
>>> A = GF(Integer(3))['t']; (t,) = A._first_ngens(1)
>>> f = A.hom([t+Integer(1)])
>>> S = OrePolynomialRing(A, f, names=('X',)); (X,) = S._first_ngens(1)
>>> P = X**Integer(2) + t
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
>>> MP = M.span(P*e0)
>>> MP2 = MP.span(P**Integer(2)*e0)

>>> MP2.is_submodule(MP)
True
>>> MP.is_submodule(MP2)
False
```

**is_zero**()

Return `True` if this Ore module is reduced to zero.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: M.is_zero()
False

sage: Q = M.quo(M)
sage: Q.is_zero()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
```

(continues on next page)

(continued from previous page)

```
>>> M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> M.is_zero()
False

>>> Q = M.quo(M)
>>> Q.is_zero()
True
```

**matrix**()

Return the matrix giving the action of the Ore variable on this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 - z^2*X + (z+2)
sage: M = S.quotient_module(P)
sage: M.matrix()
[      0       1       0]
[      0       0       1]
[4*z + 3     z^2     4*z]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) - z**Integer(2)*X + (z+Integer(2))
>>> M = S.quotient_module(P)
>>> M.matrix()
[      0       1       0]
[      0       0       1]
[4*z + 3     z^2     4*z]
```

We recognize the companion matrix attached to the Ore polynomial $P$. This is of course not a coincidence given that the pseudomorphism corresponds to the left multiplication

**See also**

*pseudohom()*

**module**()

Return the underlying free module of this Ore module.

EXAMPLES:

```
sage: A.<t> = QQ['t']
sage: S.<X> = OrePolynomialRing(A, A.derivation())
sage: M = S.quotient_module(X^3 - t)
sage: M
Ore module of rank 3 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
```

(continues on next page)

(continued from previous page)

```
sage: M.module()
Ambient free module of rank 3 over the principal ideal domain Univariate␣
↪Polynomial Ring in t over Rational Field
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> S = OrePolynomialRing(A, A.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module(X**Integer(3) - t)
>>> M
Ore module of rank 3 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt

>>> M.module()
Ambient free module of rank 3 over the principal ideal domain Univariate␣
↪Polynomial Ring in t over Rational Field
```

**multiplication_map**(*P*)

Return the multiplication by $P$ acting on this Ore module.

INPUT:

- `P` – a scalar in the base ring, or a Ore polynomial

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + a*X^2 + X - a^2
sage: M = S.quotient_module(P)
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + a*X**Integer(2) + X - a**Integer(2)
>>> M = S.quotient_module(P)
```

We define the scalar multiplication by an element in the base ring:

```
sage: f = M.multiplication_map(3)
sage: f
Ore module endomorphism of Ore module of rank 3 over Finite Field in a of␣
↪size 7^5 twisted by a |--> a^7
sage: f.matrix()
[3 0 0]
[0 3 0]
[0 0 3]
```

```
>>> from sage.all import *
>>> f = M.multiplication_map(Integer(3))
>>> f
```

(continues on next page)

(continued from previous page)

```
Ore module endomorphism of Ore module of rank 3 over Finite Field in a of␣
↪size 7^5 twisted by a |--> a^7
>>> f.matrix()
[3 0 0]
[0 3 0]
[0 0 3]
```

Be careful that an element in the base ring defines a Ore morphism if and only if it is fixed by the twisting morphisms and killed by the derivation (otherwise the multiplication by this element does not commute with the Ore action). In SageMath, attempting to create the multiplication by an element which does not fulfill these requirements leads to an error:

```
sage: M.multiplication_map(a)
Traceback (most recent call last):
...
ValueError: does not define a morphism of Ore modules
```

```
>>> from sage.all import *
>>> M.multiplication_map(a)
Traceback (most recent call last):
...
ValueError: does not define a morphism of Ore modules
```

As soon as it defines a Ore morphism, one can also build the left multiplication by an Ore polynomial:

```
sage: g = M.multiplication_map(X^5)
sage: g
Ore module endomorphism of Ore module of rank 3 over Finite Field in a of␣
↪size 7^5 twisted by a |--> a^7
sage: g.matrix()
[    3*a^4 + 3*a^3 + 6*a^2 + 5*a       4*a^4 + 5*a^3 + 2*a^2 + 6         6*a^
↪4 + 6*a^3 + a^2 + 4]
[                        a^2 + 3 5*a^4 + 5*a^3 + 6*a^2 + 4*a + 1             ␣
↪   a^3 + 5*a^2 + 4]
[6*a^4 + 6*a^3 + 3*a^2 + 3*a + 1         4*a^4 + 2*a^3 + 3*a + 5 6*a^4 + 6*a^
↪3 + 2*a^2 + 5*a + 2]
```

```
>>> from sage.all import *
>>> g = M.multiplication_map(X**Integer(5))
>>> g
Ore module endomorphism of Ore module of rank 3 over Finite Field in a of␣
↪size 7^5 twisted by a |--> a^7
>>> g.matrix()
[    3*a^4 + 3*a^3 + 6*a^2 + 5*a       4*a^4 + 5*a^3 + 2*a^2 + 6         6*a^
↪4 + 6*a^3 + a^2 + 4]
[                        a^2 + 3 5*a^4 + 5*a^3 + 6*a^2 + 4*a + 1             ␣
↪   a^3 + 5*a^2 + 4]
[6*a^4 + 6*a^3 + 3*a^2 + 3*a + 1         4*a^4 + 2*a^3 + 3*a + 5 6*a^4 + 6*a^
↪3 + 2*a^2 + 5*a + 2]
```

We check that the characteristic polynomial of $g$ is the reduced norm of the Ore polynomial $P$ we started with (this is a classical property):

```
sage: g.charpoly()
x^3 + 4*x^2 + 2*x + 5
sage: P.reduced_norm(var='x')
x^3 + 4*x^2 + 2*x + 5
```

```
>>> from sage.all import *
>>> g.charpoly()
x^3 + 4*x^2 + 2*x + 5
>>> P.reduced_norm(var='x')
x^3 + 4*x^2 + 2*x + 5
```

**ore_ring**(*names='x'*, *action=True*)

Return the underlying Ore polynomial ring.

INPUT:

- `names` (default: `x`) – a string, the name of the variable
- `action` (default: `True`) – a boolean; if `True`, an action of the Ore polynomial ring on the Ore module is set

EXAMPLES:

```
sage: K.<a> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<e1,e2> = S.quotient_module(X^2 - a)
sage: M.ore_ring()
Ore Polynomial Ring in x over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) - a, names=('e1', 'e2',)); (e1, e2,)␣
↪= M._first_ngens(2)
>>> M.ore_ring()
Ore Polynomial Ring in x over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

We can use a different variable name:

```
sage: M.ore_ring('Y')
Ore Polynomial Ring in Y over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

```
>>> from sage.all import *
>>> M.ore_ring('Y')
Ore Polynomial Ring in Y over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

Alternatively, one can use the following shortcut:

```
sage: T.<Z> = M.ore_ring()
sage: T
Ore Polynomial Ring in Z over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

```
>>> from sage.all import *
>>> T = M.ore_ring(names=('Z',)); (Z,) = T._first_ngens(1)
>>> T
Ore Polynomial Ring in Z over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

In all the above cases, an action of the returned Ore polynomial ring on $M$ is registered:

```
sage: Z*e1
e2
sage: Z*e2
a*e1
```

```
>>> from sage.all import *
>>> Z*e1
e2
>>> Z*e2
a*e1
```

Specifying `action=False` prevents this to happen:

```
sage: T.<U> = M.ore_ring(action=False)
sage: U*e1
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
    'Ore Polynomial Ring in U over Finite Field in a of size 5^3 twisted by a␣
↪|--> a^5' and
    'Ore module <e1, e2> over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5'
```

```
>>> from sage.all import *
>>> T = M.ore_ring(action=False, names=('U',)); (U,) = T._first_ngens(1)
>>> U*e1
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
    'Ore Polynomial Ring in U over Finite Field in a of size 5^3 twisted by a␣
↪|--> a^5' and
    'Ore module <e1, e2> over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5'
```

**over_fraction_field**()

Return the scalar extension of this Ore module to the fraction field.

EXAMPLES:

```
sage: A.<t> = QQ[]
sage: d = A.derivation()
sage: S.<X> = OrePolynomialRing(A, d)
sage: M = S.quotient_module(X^2 + t*X + t)
sage: M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
sage: M.over_fraction_field()
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> d = A.derivation()
>>> S = OrePolynomialRing(A, d, names=('X',)); (X,) = S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t*X + t)
>>> M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> M.over_fraction_field()
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

If given, the variable names are preserved in this operation:

```
sage: N.<u,v> = S.quotient_module(X^2 + t*X + t)
sage: N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
sage: N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(2) + t*X + t, names=('u', 'v',)); (u, v,
↪) = N._first_ngens(2)
>>> N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

When the base ring is already a field, the same module is returned:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z*X + z)
sage: M.over_fraction_field() is M
True
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z*X + z)
>>> M.over_fraction_field() is M
True
```

**pseudohom**()

Return the pseudomorphism giving the action of the Ore variable on this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 - z^2*X + (z+2)
sage: M = S.quotient_module(P)
sage: M.pseudohom()
Free module pseudomorphism (twisted by z |--> z^5) defined by the matrix
[      0       1       0]
[      0       0       1]
[4*z + 3     z^2     4*z]
Domain: Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
Codomain: Ore module of rank 3 over Finite Field in z of size 5^3 twisted by␣
↪z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) - z**Integer(2)*X + (z+Integer(2))
>>> M = S.quotient_module(P)
>>> M.pseudohom()
Free module pseudomorphism (twisted by z |--> z^5) defined by the matrix
[      0       1       0]
[      0       0       1]
[4*z + 3     z^2     4*z]
Domain: Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
Codomain: Ore module of rank 3 over Finite Field in z of size 5^3 twisted by␣
↪z |--> z^5
```

**See also**

*matrix()*

**quo**(*sub*, *remove_torsion=False*, *names=None*, *check=True*)

alias of *quotient()*.

**quotient**(*sub*, *remove_torsion=False*, *names=None*, *check=True*)

Return the quotient of this Ore module by the submodule generated (over the underlying Ore ring) by `gens`.

INPUT:

- `sub` – a list of vectors or submodules of this Ore module
- `remove_torsion` (default: `False`) – a boolean
- `names` (default: `None`) – the name of the vectors in a basis of the quotient
- `check` (default: `True`) – a boolean, ignored

EXAMPLES:

```
sage: A.<t> = GF(5)['t']
sage: S.<X> = OrePolynomialRing(A, A.derivation())
sage: P = X^2 + t*X + t
sage: M = S.quotient_module(P^3, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> A = GF(Integer(5))['t']; (t,) = A._first_ngens(1)
>>> S = OrePolynomialRing(A, A.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> P = X**Integer(2) + t*X + t
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

We create the quotient $M/MP$:

```
sage: modP = M.quotient(P*e0)
sage: modP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> modP = M.quotient(P*e0)
>>> modP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

As a shortcut, we can write `quo` instead of `quotient` or even use the `/` operator:

```
sage: modP = M / (P*e0)
sage: modP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> modP = M / (P*e0)
>>> modP
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

In the above example, the quotient is still a free module. It might happen however that torsion shows up in the quotient. Currently, torsion Ore modules are not implemented, so attempting to create a quotient with

torsion raises an error:

```
sage: M.quotient(X^5*e0)
Traceback (most recent call last):
...
NotImplementedError: torsion Ore modules are not implemented
```

```
>>> from sage.all import *
>>> M.quotient(X**Integer(5)*e0)
Traceback (most recent call last):
...
NotImplementedError: torsion Ore modules are not implemented
```

It is nevertheless always possible to build the free part of the quotient by passing in the argument `remove_torsion=True`:

```
sage: M.quotient(X^5*e0, remove_torsion=True)
Ore module of rank 0 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> M.quotient(X**Integer(5)*e0, remove_torsion=True)
Ore module of rank 0 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
```

By default, the vectors in the quotient have the same names as their representatives in $M$:

```
sage: modP.basis()
[(t+4)*e0 + (t+3)*e1, (4*t+3)*e0 + t*e2]
```

```
>>> from sage.all import *
>>> modP.basis()
[(t+4)*e0 + (t+3)*e1, (4*t+3)*e0 + t*e2]
```

One can override this behavior by setting the attributes `names`:

```
sage: modP = M.quo(P*e0, names='u')
sage: modP.inject_variables()
Defining u0, u1
sage: modP.basis()
[u0, u1]
```

```
>>> from sage.all import *
>>> modP = M.quo(P*e0, names='u')
>>> modP.inject_variables()
Defining u0, u1
>>> modP.basis()
[u0, u1]
```

Note that a coercion map from the initial Ore module to its quotient is automatically set. As a consequence, combining elements of `M` and `modP` in the same formula works:

```
sage: t*u0 + e1
(t^2+2*t+2)*u0 + (t+4)*u1
```

```
>>> from sage.all import *
>>> t*u0 + e1
(t^2+2*t+2)*u0 + (t+4)*u1
```

One can combine the construction of quotients and submodules without trouble. For instance, here we build the space $MP/MP^2$:

```
sage: modP2 = M / (P^2*e0)
sage: N = modP2.span(P*e0)
sage: N
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
sage: N.basis()
[t*e0 + t*e1 + e2, (4*t+1)*e0 + e1 + (t+4)*e2 + e3]
```

```
>>> from sage.all import *
>>> modP2 = M / (P**Integer(2)*e0)
>>> N = modP2.span(P*e0)
>>> N
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
>>> N.basis()
[t*e0 + t*e1 + e2, (4*t+1)*e0 + e1 + (t+4)*e2 + e3]
```

**See also**

*quo()*, *span()*

**random_element**(**args*, ***kwds*)

Return a random element in this Ore module.

Extra arguments are passed to the random generator of the base ring.

EXAMPLES:

```
sage: A.<t> = QQ['t']
sage: S.<X> = OrePolynomialRing(A, A.derivation())
sage: M = S.quotient_module(X^3 - t, names='e')
sage: M.random_element()    # random
(-1/2*t^2 - 3/4*t + 3/2)*e0 + (-3/2*t^2 - 3*t + 4)*e1 + (-6*t + 2)*e2

sage: M.random_element(degree=5)    # random
(4*t^5 - 1/2*t^4 + 3/2*t^3 + 6*t^2 - t - 1/10)*e0 + (19/3*t^5 - t^3 - t^2 +␣
↪1)*e1 + (t^5 + 4*t^4 + 4*t^2 + 1/3*t - 33)*e2
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> S = OrePolynomialRing(A, A.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module(X**Integer(3) - t, names='e')
>>> M.random_element()    # random
(-1/2*t^2 - 3/4*t + 3/2)*e0 + (-3/2*t^2 - 3*t + 4)*e1 + (-6*t + 2)*e2
```

(continues on next page)

(continued from previous page)

```
>>> M.random_element(degree=Integer(5))    # random
(4*t^5 - 1/2*t^4 + 3/2*t^3 + 6*t^2 - t - 1/10)*e0 + (19/3*t^5 - t^3 - t^2 +␣
↪1)*e1 + (t^5 + 4*t^4 + 4*t^2 + 1/3*t - 33)*e2
```

**rename_basis**(*names*, *coerce=False*)

Return the same Ore module with the given naming for the vectors in its distinguished basis.

INPUT:

- `names` – a string or a list of strings, the new names
- `coerce` (default: `False`) – a boolean; if `True`, a coercion map from this Ore module to renamed version is set

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5

sage: Me = M.rename_basis('e')
sage: Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
>>> M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5

>>> Me = M.rename_basis('e')
>>> Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

Now compare how elements are displayed:

```
sage: M.random_element()    # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
sage: Me.random_element()  # random
(2*z+4)*e0 + (z^2+4*z+4)*e1
```

```
>>> from sage.all import *
>>> M.random_element()    # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
>>> Me.random_element()  # random
(2*z+4)*e0 + (z^2+4*z+4)*e1
```

At this point, there is no coercion map between `M` and `Me`. Therefore, adding elements in both parents results in an error:

```
sage: M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

```
>>> from sage.all import *
>>> M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

In order to set this coercion, one should define `Me` by passing the extra argument `coerce=True`:

```
sage: Me = M.rename_basis('e', coerce=True)
sage: M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2+z+4)*e1
```

```
>>> from sage.all import *
>>> Me = M.rename_basis('e', coerce=True)
>>> M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2+z+4)*e1
```

**Warning**

Use `coerce=True` with extreme caution. Indeed, setting inappropriate coercion maps may result in a circular path in the coercion graph which, in turn, could eventually break the coercion system.

Note that the bracket construction also works:

```
sage: M.<v,w> = M.rename_basis()
sage: M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> M = M.rename_basis(names=('v', 'w',)); (v, w,) = M._first_ngens(2)
>>> M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

In this case, $v$ and $w$ are automatically defined:

```
sage: v + w
v + w
```

```
>>> from sage.all import *
>>> v + w
v + w
```

**span**(*gens*, *saturate=False*, *names=None*, *check=True*)

Return the submodule or saturated submodule of this Ore module generated (over the underlying Ore ring) by `gens`.

We recall that a submodule $N$ of $M$ is called saturated if the quotient $M/N$ has no torsion. The saturation of $N$ in $M$ is the submodule $N' \subset M$ consisting of vectors $x \in M$ such that $ax \in N$ for some nonzero $a$ in the base ring.

INPUT:

- `gens` – a list of vectors or submodules of this Ore module
- `saturate` (default: `False`) – a boolean; if `True`, return the saturation of the submodule generated by `gens`
- `names` (default: `None`) – the name of the vectors in a basis of this submodule
- `check` (default: `True`) – a boolean, ignored

EXAMPLES:

```
sage: A.<t> = GF(5)['t']
sage: S.<X> = OrePolynomialRing(A, A.derivation())
sage: P = X^2 + t*X + t
sage: M = S.quotient_module(P^3, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> A = GF(Integer(5))['t']; (t,) = A._first_ngens(1)
>>> S = OrePolynomialRing(A, A.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> P = X**Integer(2) + t*X + t
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

We create the submodule $MP$:

```
sage: MP = M.span([P*e0])
sage: MP
Ore module of rank 4 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
sage: MP.basis()
[t*e0 + t*e1 + e2,
 (4*t+1)*e0 + e1 + (t+4)*e2 + e3,
 (t+4)*e0 + e1 + 3*e2 + (t+4)*e3 + e4,
 (4*t+1)*e0 + 4*e1 + 4*e3 + (t+4)*e4 + e5]
```

```
>>> from sage.all import *
>>> MP = M.span([P*e0])
>>> MP
```

(continues on next page)

(continued from previous page)

```
Ore module of rank 4 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
>>> MP.basis()
[t*e0 + t*e1 + e2,
 (4*t+1)*e0 + e1 + (t+4)*e2 + e3,
 (t+4)*e0 + e1 + 3*e2 + (t+4)*e3 + e4,
 (4*t+1)*e0 + 4*e1 + 4*e3 + (t+4)*e4 + e5]
```

When there is only one generator, encapsulating it in a list is not necessary; one can equally write:

```
sage: MP = M.span(P*e0)
```

```
>>> from sage.all import *
>>> MP = M.span(P*e0)
```

In this case, the module $MP$ is already saturated, so computing its saturation yields the same result:

```
sage: MPsat = M.span(P*e0, saturate=True)
sage: MPsat.basis()
[t*e0 + t*e1 + e2,
 (4*t+1)*e0 + e1 + (t+4)*e2 + e3,
 (t+4)*e0 + e1 + 3*e2 + (t+4)*e3 + e4,
 (4*t+1)*e0 + 4*e1 + 4*e3 + (t+4)*e4 + e5]
sage: MPsat == MP
True
```

```
>>> from sage.all import *
>>> MPsat = M.span(P*e0, saturate=True)
>>> MPsat.basis()
[t*e0 + t*e1 + e2,
 (4*t+1)*e0 + e1 + (t+4)*e2 + e3,
 (t+4)*e0 + e1 + 3*e2 + (t+4)*e3 + e4,
 (4*t+1)*e0 + 4*e1 + 4*e3 + (t+4)*e4 + e5]
>>> MPsat == MP
True
```

Of course, it is not always the case:

```
sage: N = M.span(X^5*e0)
sage: N.basis()
[(t^3+4*t^2+4*t)*e0 + (t^2+3*t+3)*e1 + (2*t^2+t+4)*e2 + 3*t^2*e3 + (2*t^
↪2+2*t+2)*e4,
 (3*t^2+3*t+4)*e0 + (t^3+4*t^2+t+3)*e1 + (t^2+2*t+4)*e2 + (2*t^2+2*t+4)*e3 +␣
↪(3*t^2+4*t+2)*e4,
 (t+3)*e0 + (t^2+t)*e1 + (t^3+4*t^2+3*t)*e2 + (t^2+t+1)*e3 + (2*t^2+3*t+3)*e4,
 e0 + (3*t+4)*e1 + (4*t^2+4*t+3)*e2 + (t^3+4*t^2+1)*e3 + (t^2+4)*e4,
 4*e1 + (t+3)*e2 + (2*t^2+2*t+3)*e3 + (t^3+4*t^2+2*t+1)*e4,
 e5]
sage: Nsat = M.span(X^5*e0, saturate=True)
sage: Nsat.basis()
[e0, e1, e2, e3, e4, e5]
```

```
>>> from sage.all import *
>>> N = M.span(X**Integer(5)*e0)
>>> N.basis()
[(t^3+4*t^2+4*t)*e0 + (t^2+3*t+3)*e1 + (2*t^2+t+4)*e2 + 3*t^2*e3 + (2*t^
↪2+2*t+2)*e4,
 (3*t^2+3*t+4)*e0 + (t^3+4*t^2+t+3)*e1 + (t^2+2*t+4)*e2 + (2*t^2+2*t+4)*e3 +␣
↪(3*t^2+4*t+2)*e4,
 (t+3)*e0 + (t^2+t)*e1 + (t^3+4*t^2+3*t)*e2 + (t^2+t+1)*e3 + (2*t^2+3*t+3)*e4,
 e0 + (3*t+4)*e1 + (4*t^2+4*t+3)*e2 + (t^3+4*t^2+1)*e3 + (t^2+4)*e4,
 4*e1 + (t+3)*e2 + (2*t^2+2*t+3)*e3 + (t^3+4*t^2+2*t+1)*e4,
 e5]
>>> Nsat = M.span(X**Integer(5)*e0, saturate=True)
>>> Nsat.basis()
[e0, e1, e2, e3, e4, e5]
```

If one wants, one can give names to the basis of the submodule using the attribute `names`:

```
sage: MP2 = M.span(P^2*e0, names='u')
sage: MP2.inject_variables()
Defining u0, u1
sage: MP2.basis()
[u0, u1]

sage: M(u0)
(t^2+t)*e0 + (2*t^2+t+2)*e1 + (t^2+2*t+2)*e2 + 2*t*e3 + e4
```

```
>>> from sage.all import *
>>> MP2 = M.span(P**Integer(2)*e0, names='u')
>>> MP2.inject_variables()
Defining u0, u1
>>> MP2.basis()
[u0, u1]

>>> M(u0)
(t^2+t)*e0 + (2*t^2+t+2)*e1 + (t^2+2*t+2)*e2 + 2*t*e3 + e4
```

Note that a coercion map from the submodule to the ambient module is automatically set:

```
sage: M.has_coerce_map_from(MP2)
True
```

```
>>> from sage.all import *
>>> M.has_coerce_map_from(MP2)
True
```

Therefore, combining elements of `M` and `MP2` in the same expression perfectly works:

```
sage: t*u0 + e1
(t^3+t^2)*e0 + (2*t^3+t^2+2*t+1)*e1 + (t^3+2*t^2+2*t)*e2 + 2*t^2*e3 + t*e4
```

```
>>> from sage.all import *
>>> t*u0 + e1
(t^3+t^2)*e0 + (2*t^3+t^2+2*t+1)*e1 + (t^3+2*t^2+2*t)*e2 + 2*t^2*e3 + t*e4
```

Here is an example with multiple generators:

```
sage: MM = M.span([MP2, P*e1])
sage: MM.basis()
[e0, e1, e2, e3, e4, e5]
```

```
>>> from sage.all import *
>>> MM = M.span([MP2, P*e1])
>>> MM.basis()
[e0, e1, e2, e3, e4, e5]
```

In this case, we obtain the whole space.

Creating submodules of submodules is also allowed:

```
sage: N = MP.span(P^2*e0)
sage: N
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
sage: N.basis()
[(t^2+t)*e0 + (2*t^2+t+2)*e1 + (t^2+2*t+2)*e2 + 2*t*e3 + e4,
 (3*t^2+1)*e0 + (2*t^2+3*t+2)*e1 + 4*t*e2 + (t^2+3*t+4)*e3 + (2*t+3)*e4 + e5]
```

```
>>> from sage.all import *
>>> N = MP.span(P**Integer(2)*e0)
>>> N
Ore module of rank 2 over Univariate Polynomial Ring in t over Finite Field␣
↪of size 5 twisted by d/dt
>>> N.basis()
[(t^2+t)*e0 + (2*t^2+t+2)*e1 + (t^2+2*t+2)*e2 + 2*t*e3 + e4,
 (3*t^2+1)*e0 + (2*t^2+3*t+2)*e1 + 4*t*e2 + (t^2+3*t+4)*e3 + (2*t+3)*e4 + e5]
```

**See also**

*quotient()*

**submodule**(*gens*, *saturate=False*, *names=None*, *check=True*)

alias of *span()*.

**twisting_derivation**()

Return the twisting derivation corresponding to this Ore module.

EXAMPLES:

```
sage: R.<t> = QQ[]
sage: T.<Y> = OrePolynomialRing(R, R.derivation())
sage: M = T.quotient_module(Y + t^2)
sage: M.twisting_derivation()
d/dt
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> T = OrePolynomialRing(R, R.derivation(), names=('Y',)); (Y,) = T._first_
```

(continues on next page)

(continued from previous page)

```
↪ngens(1)
>>> M = T.quotient_module(Y + t**Integer(2))
>>> M.twisting_derivation()
d/dt
```

When the twisting derivation in zero, nothing is returned:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X + z)
sage: M.twisting_derivation()
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X + z)
>>> M.twisting_derivation()
```

**See also**

*twisting_morphism()*

**twisting_morphism**()

Return the twisting morphism corresponding to this Ore module.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X + z)
sage: M.twisting_morphism()
Frobenius endomorphism z |--> z^5 on Finite Field in z of size 5^3
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X + z)
>>> M.twisting_morphism()
Frobenius endomorphism z |--> z^5 on Finite Field in z of size 5^3
```

When the twisting morphism is trivial (that is, the identity), nothing is returned:

```
sage: R.<t> = QQ[]
sage: T.<Y> = OrePolynomialRing(R, R.derivation())
sage: M = T.quotient_module(Y + t^2)
sage: M.twisting_morphism()
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> T = OrePolynomialRing(R, R.derivation(), names=('Y',)); (Y,) = T._first_
↪ngens(1)
>>> M = T.quotient_module(Y + t**Integer(2))
>>> M.twisting_morphism()
```

**See also**

*twisting_derivation()*

**class** sage.modules.ore_module.**OreQuotientModule**(*cover*, *submodule*, *names*)

Bases: *OreModule*

Class for quotients of Ore modules.

**cover**()

If this quotient in $M/N$, return $M$.

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: M.<v,w> = S.quotient_module((X + t)^2)
sage: N = M.quo((X + t)*v)

sage: N.cover()
Ore module <v, w> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Finite Field of size 5 twisted by d/dt
sage: N.cover() is M
True
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module((X + t)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> N = M.quo((X + t)*v)

>>> N.cover()
Ore module <v, w> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Finite Field of size 5 twisted by d/dt
>>> N.cover() is M
True
```

**See also**

*relations()*

**covers**()

Return the list of modules of which this module is a quotient.

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^2 + a
sage: M = S.quotient_module(P^3, names='e')
sage: M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(2) + a
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

For an ambient module, the list is reduced to one element (namely the module itself):

```
sage: M.covers()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> M.covers()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We now create a quotient of $M$ and observe what happens:

```
sage: MP2 = M.quo(P^2*e0)
sage: MP2.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = M.quo(P**Integer(2)*e0)
>>> MP2.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

If we now create a quotient of $M/MP$, another item is added to the list:

```
sage: MP = MP2.quo(P*e0)
sage: MP.covers()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = MP2.quo(P*e0)
>>> MP.covers()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We underline nevertheless that if we directly define $M/MP$ has a quotient of $M$, the intermediate $M/MP^2$ does not show up in the list:

```
sage: MP = M.quo(P^2*e0)
sage: MP.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = M.quo(P**Integer(2)*e0)
>>> MP.covers()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

**morphism_modulo**($f$)

If this quotient in $M/N$ and $f : X \to M$ is a morphism, return the induced map $X \to M/N$.

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: P = X + t
sage: M.<v,w> = S.quotient_module(P^2)
sage: Q.<wbar> = M.quo(P*v)

sage: f = M.multiplication_map(X^5)
sage: f
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
sage: g = Q.morphism_modulo(f)
```

(continues on next page)

(continued from previous page)

```
sage: g
Ore module morphism:
  From: Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <wbar> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> P = X + t
>>> M = S.quotient_module(P**Integer(2), names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> Q = M.quo(P*v, names=('wbar',)); (wbar,) = Q._first_ngens(1)

>>> f = M.multiplication_map(X**Integer(5))
>>> f
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
>>> g = Q.morphism_modulo(f)
>>> g
Ore module morphism:
  From: Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <wbar> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
```

**morphism_quotient**($f$)

If this quotient in $M/N$ and $f : M \to X$ is a morphism vanishing on $N$, return the induced map $M/N \to X$.

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: P = X + t
sage: M.<v,w> = S.quotient_module(P^2)
sage: Q.<wbar> = M.quo(P*v)

sage: f = M.hom({v: P*v})
sage: f
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
sage: g = Q.morphism_quotient(f)
sage: g
Ore module morphism:
  From: Ore module <wbar> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> P = X + t
>>> M = S.quotient_module(P**Integer(2), names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> Q = M.quo(P*v, names=('wbar',)); (wbar,) = Q._first_ngens(1)

>>> f = M.hom({v: P*v})
>>> f
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
>>> g = Q.morphism_quotient(f)
>>> g
Ore module morphism:
  From: Ore module <wbar> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
```

When the given morphism does not vanish on $N$, an error is raised:

```
sage: h = M.multiplication_map(X^5)
sage: h
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
sage: Q.morphism_quotient(h)
Traceback (most recent call last):
...
ValueError: the morphism does not factor through this quotient
```

```
>>> from sage.all import *
>>> h = M.multiplication_map(X**Integer(5))
>>> h
Ore module endomorphism of Ore module <v, w> over Fraction Field of␣
↪Univariate Polynomial Ring in t over Finite Field of size 5 twisted by d/dt
>>> Q.morphism_quotient(h)
Traceback (most recent call last):
...
ValueError: the morphism does not factor through this quotient
```

**over_fraction_field**()

Return the scalar extension of this Ore module to the fraction field.

EXAMPLES:

```
sage: A.<t> = QQ[]
sage: d = A.derivation()
sage: S.<X> = OrePolynomialRing(A, d)
sage: M = S.quotient_module(X^2 + t*X + t)
sage: M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
```

(continues on next page)

(continued from previous page)

```
↪twisted by d/dt
sage: M.over_fraction_field()
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> d = A.derivation()
>>> S = OrePolynomialRing(A, d, names=('X',)); (X,) = S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t*X + t)
>>> M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> M.over_fraction_field()
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

If given, the variable names are preserved in this operation:

```
sage: N.<u,v> = S.quotient_module(X^2 + t*X + t)
sage: N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
sage: N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(2) + t*X + t, names=('u', 'v',)); (u, v,
↪) = N._first_ngens(2)
>>> N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

When the base ring is already a field, the same module is returned:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z*X + z)
sage: M.over_fraction_field() is M
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z*X + z)
>>> M.over_fraction_field() is M
True
```

**projection_morphism**()

Return the projection from the cover module to this quotient.

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: M.<v,w> = S.quotient_module((X + t)^2)
sage: Q = M.quo((X + t)*v)
sage: Q.projection_morphism()
Ore module morphism:
  From: Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module of rank 1 over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module((X + t)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> Q = M.quo((X + t)*v)
>>> Q.projection_morphism()
Ore module morphism:
  From: Ore module <v, w> over Fraction Field of Univariate Polynomial Ring␣
↪in t over Finite Field of size 5 twisted by d/dt
  To:   Ore module of rank 1 over Fraction Field of Univariate Polynomial␣
↪Ring in t over Finite Field of size 5 twisted by d/dt
```

**relations**(*names=None*)

If this quotient in $M/N$, return $N$.

INPUT:

- `names` – the names of the vectors of the basis of $N$, or `None`

EXAMPLES:

```
sage: K.<t> = Frac(GF(5)['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: M.<v,w> = S.quotient_module((X + t)^2)
sage: Q = M.quo((X + t)*v)

sage: N = Q.relations()
sage: N
Ore module of rank 1 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Finite Field of size 5 twisted by d/dt
sage: (X + t)*v in N
True
sage: Q == M/N
True
```

```
>>> from sage.all import *
>>> K = Frac(GF(Integer(5))['t'], names=('t',)); (t,) = K._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module((X + t)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> Q = M.quo((X + t)*v)

>>> N = Q.relations()
>>> N
Ore module of rank 1 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Finite Field of size 5 twisted by d/dt
>>> (X + t)*v in N
True
>>> Q == M/N
True
```

It is also possible to define names for the basis elements of $N$:

```
sage: N.<u> = Q.relations()
sage: N
Ore module <u> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Finite Field of size 5 twisted by d/dt
sage: M(u)
v + 1/t*w
```

```
>>> from sage.all import *
>>> N = Q.relations(names=('u',)); (u,) = N._first_ngens(1)
>>> N
Ore module <u> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Finite Field of size 5 twisted by d/dt
>>> M(u)
v + 1/t*w
```

**See also**

*relations()*

**rename_basis**(*names*, *coerce=False*)

Return the same Ore module with the given naming for the vectors in its distinguished basis.

INPUT:

- `names` – a string or a list of strings, the new names
- `coerce` (default: `False`) – a boolean; if `True`, a coercion map from this Ore module to the renamed version is set

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z*X + 1)
sage: M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

(continues on next page)

(continued from previous page)

```
sage: Me = M.rename_basis('e')
sage: Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z*X + Integer(1))
>>> M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5

>>> Me = M.rename_basis('e')
>>> Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

Now compare how elements are displayed:

```
sage: M.random_element()    # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
sage: Me.random_element()  # random
(2*z + 4)*e0 + (z^2 + 4*z + 4)*e1
```

```
>>> from sage.all import *
>>> M.random_element()    # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
>>> Me.random_element()  # random
(2*z + 4)*e0 + (z^2 + 4*z + 4)*e1
```

At this point, there is no coercion map between `M` and `Me`. Therefore, adding elements in both parents results in an error:

```
sage: M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

```
>>> from sage.all import *
>>> M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

In order to set this coercion, one should define `Me` by passing the extra argument `coerce=True`:

```
sage: Me = M.rename_basis('e', coerce=True)
sage: M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2 + z + 4)*e1
```

```
>>> from sage.all import *
>>> Me = M.rename_basis('e', coerce=True)
>>> M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2 + z + 4)*e1
```

**Warning**

Use `coerce=True` with extreme caution. Indeed, setting inappropriate coercion maps may result in a circular path in the coercion graph which, in turn, could eventually break the coercion system.

Note that the bracket construction also works:

```
sage: M.<v,w> = M.rename_basis()
sage: M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> M = M.rename_basis(names=('v', 'w',)); (v, w,) = M._first_ngens(2)
>>> M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

In this case, $v$ and $w$ are automatically defined:

```
sage: v + w
v + w
```

```
>>> from sage.all import *
>>> v + w
v + w
```

**class** sage.modules.ore_module.**OreSubmodule**(*ambient*, *submodule*, *names*)

Bases: *OreModule*

Class for submodules of Ore modules.

**ambient_module**()

Return the ambient Ore module in which this submodule lives.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<v,w> = S.quotient_module((X + z)^2)
sage: N = M.span((X + z)*v)
sage: N.ambient_module()
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: N.ambient_module() is M
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module((X + z)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> N = M.span((X + z)*v)
>>> N.ambient_module()
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> N.ambient_module() is M
True
```

**ambient_modules**()

Return the list of modules in which this module naturally lives.

EXAMPLES:

```
sage: K.<a> = GF(7^5)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^2 + a
sage: M = S.quotient_module(P^3, names='e')
sage: M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(5), names=('a',)); (a,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(2) + a
>>> M = S.quotient_module(P**Integer(3), names='e')
>>> M
Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5
```

For an ambient module, the list is reduced to one element (namely the module itself):

```
sage: M.ambient_modules()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> M.ambient_modules()
[Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

On the contrary, for a submodule of $M$, the list also contains the ambient space:

```
sage: MP = M.span(P*e0)
sage: MP.ambient_modules()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP = M.span(P*e0)
>>> MP.ambient_modules()
[Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

If we now create a submodule of $MP$, the list gets even longer:

```
sage: MP2 = MP.span(P^2*e0)
sage: MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = MP.span(P**Integer(2)*e0)
>>> MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module of rank 4 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

We underline nevertheless that if we define $MP^2$ has a submodule of $M$, the intermediate $MP$ does not show up in the list:

```
sage: MP2 = M.span(P^2*e0)
sage: MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

```
>>> from sage.all import *
>>> MP2 = M.span(P**Integer(2)*e0)
>>> MP2.ambient_modules()
[Ore module of rank 2 over Finite Field in a of size 7^5 twisted by a |--> a^
↪7,
 Ore module <e0, e1, e2, e3, e4, e5> over Finite Field in a of size 7^5␣
↪twisted by a |--> a^7]
```

**injection_morphism**()

Return the inclusion of this submodule in the ambient space.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<v,w> = S.quotient_module((X + z)^2)
sage: N = M.span((X + z)*v)
sage: N.injection_morphism()
Ore module morphism:
  From: Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
  To:   Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module((X + z)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> N = M.span((X + z)*v)
>>> N.injection_morphism()
Ore module morphism:
  From: Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
  To:   Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
```

**morphism_corestriction**($f$)

If the image of $f$ is contained in this submodule, return the corresponding corestriction of $f$.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X + z
sage: M.<v,w> = S.quotient_module(P^2)
sage: N = M.span(P*v)

sage: f = M.hom({v: P*v})
sage: f
Ore module endomorphism of Ore module <v, w> over Finite Field in z of size 5^
↪3 twisted by z |--> z^5

sage: g = N.morphism_corestriction(f)
sage: g
Ore module morphism:
  From: Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
  To:   Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
sage: g.matrix()
```

(continues on next page)

(continued from previous page)

```
[    z]
[4*z^2]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X + z
>>> M = S.quotient_module(P**Integer(2), names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> N = M.span(P*v)

>>> f = M.hom({v: P*v})
>>> f
Ore module endomorphism of Ore module <v, w> over Finite Field in z of size 5^
↪3 twisted by z |--> z^5

>>> g = N.morphism_corestriction(f)
>>> g
Ore module morphism:
  From: Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
  To:   Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
>>> g.matrix()
[    z]
[4*z^2]
```

When the image of the morphism is not contained in this submodule, an error is raised:

```
sage: h = M.multiplication_map(X^3)
sage: N.morphism_corestriction(h)
Traceback (most recent call last):
...
ValueError: the image of the morphism is not contained in this submodule
```

```
>>> from sage.all import *
>>> h = M.multiplication_map(X**Integer(3))
>>> N.morphism_corestriction(h)
Traceback (most recent call last):
...
ValueError: the image of the morphism is not contained in this submodule
```

**morphism_restriction**($f$)

Return the restriction of $f$ to this submodule.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<v,w> = S.quotient_module((X + z)^2)
sage: N = M.span((X + z)*v)
```

(continues on next page)

(continued from previous page)

```
sage: f = M.multiplication_map(X^3)
sage: f
Ore module endomorphism of Ore module <v, w> over Finite Field in z of size 5^
↪3 twisted by z |--> z^5

sage: g = N.morphism_restriction(f)
sage: g
Ore module morphism:
  From: Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
  To:   Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
sage: g.matrix()
[        3 4*z^2 + 2]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module((X + z)**Integer(2), names=('v', 'w',)); (v, w,) =␣
↪M._first_ngens(2)
>>> N = M.span((X + z)*v)

>>> f = M.multiplication_map(X**Integer(3))
>>> f
Ore module endomorphism of Ore module <v, w> over Finite Field in z of size 5^
↪3 twisted by z |--> z^5

>>> g = N.morphism_restriction(f)
>>> g
Ore module morphism:
  From: Ore module of rank 1 over Finite Field in z of size 5^3 twisted by z␣
↪|--> z^5
  To:   Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |-->
↪ z^5
>>> g.matrix()
[        3 4*z^2 + 2]
```

**over_fraction_field**()

Return the scalar extension of this Ore module to the fraction field.

EXAMPLES:

```
sage: A.<t> = QQ[]
sage: d = A.derivation()
sage: S.<X> = OrePolynomialRing(A, d)
sage: M = S.quotient_module(X^2 + t*X + t)
sage: M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
sage: M.over_fraction_field()
```

(continues on next page)

(continued from previous page)

```
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> d = A.derivation()
>>> S = OrePolynomialRing(A, d, names=('X',)); (X,) = S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t*X + t)
>>> M
Ore module of rank 2 over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> M.over_fraction_field()
Ore module of rank 2 over Fraction Field of Univariate Polynomial Ring in t␣
↪over Rational Field twisted by d/dt
```

If given, the variable names are preserved in this operation:

```
sage: N.<u,v> = S.quotient_module(X^2 + t*X + t)
sage: N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
sage: N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(2) + t*X + t, names=('u', 'v',)); (u, v,
↪) = N._first_ngens(2)
>>> N
Ore module <u, v> over Univariate Polynomial Ring in t over Rational Field␣
↪twisted by d/dt
>>> N.over_fraction_field()
Ore module <u, v> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
```

When the base ring is already a field, the same module is returned:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z*X + z)
sage: M.over_fraction_field() is M
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z*X + z)
>>> M.over_fraction_field() is M
True
```

**rename_basis**(*names*, *coerce=False*)

Return the same Ore module with the given naming for the vectors in its distinguished basis.

INPUT:

- `names` – a string or a list of strings, the new names
- `coerce` (default: `False`) – a boolean; if `True`, a coercion map from this Ore module to renamed version is set

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z^2)
sage: M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5

sage: Me = M.rename_basis('e')
sage: Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z**Integer(2))
>>> M
Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5

>>> Me = M.rename_basis('e')
>>> Me
Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

Now compare how elements are displayed:

```
sage: M.random_element()   # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
sage: Me.random_element()  # random
(2*z + 4)*e0 + (z^2 + 4*z + 4)*e1
```

```
>>> from sage.all import *
>>> M.random_element()   # random
(3*z^2 + 4*z + 2, 3*z^2 + z)
>>> Me.random_element()  # random
(2*z + 4)*e0 + (z^2 + 4*z + 4)*e1
```

At this point, there is no coercion map between `M` and `Me`. Therefore, adding elements in both parents results in an error:

```
sage: M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

(continues on next page)

(continued from previous page)

```
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

```
>>> from sage.all import *
>>> M.random_element() + Me.random_element()
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ore module of rank 2 over Finite Field in z of size 5^3 twisted by z |--> z^5
↪' and
'Ore module <e0, e1> over Finite Field in z of size 5^3 twisted by z |--> z^5'
```

In order to set this coercion, one should define `Me` by passing the extra argument `coerce=True`:

```
sage: Me = M.rename_basis('e', coerce=True)
sage: M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2 + z + 4)*e1
```

```
>>> from sage.all import *
>>> Me = M.rename_basis('e', coerce=True)
>>> M.random_element() + Me.random_element()  # random
2*z^2*e0 + (z^2 + z + 4)*e1
```

**Warning**

Use `coerce=True` with extreme caution. Indeed, setting inappropriate coercion maps may result in a circular path in the coercion graph which, in turn, could eventually break the coercion system.

Note that the bracket construction also works:

```
sage: M.<v,w> = M.rename_basis()
sage: M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> M = M.rename_basis(names=('v', 'w',)); (v, w,) = M._first_ngens(2)
>>> M
Ore module <v, w> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

In this case, $v$ and $w$ are automatically defined:

```
sage: v + w
v + w
```

```
>>> from sage.all import *
>>> v + w
v + w
```

**saturate**(*names=None*, *coerce=False*)

Return the saturation of this module in the ambient module.

By definition, the saturation of $N$ in $M$ is the submodule of $M$ consisting of vectors $x$ such that $ax \in N$ for a nonzero scalar $a$ in the base ring.

INPUT:

- `names` – a string or a list of strings, the names of the vectors in a basis of the saturation
- `coerce` (default: `False`) – a boolean; if `True`, a coercion map from this Ore module to its saturation is set

EXAMPLES:

```
sage: A.<t> = GF(3)[]
sage: f = A.hom([t+1])
sage: S.<X> = OrePolynomialRing(A, f)
sage: P = X^2 + t
sage: M = S.quotient_module(P^2, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3
```

```
>>> from sage.all import *
>>> A = GF(Integer(3))['t']; (t,) = A._first_ngens(1)
>>> f = A.hom([t+Integer(1)])
>>> S = OrePolynomialRing(A, f, names=('X',)); (X,) = S._first_ngens(1)
>>> P = X**Integer(2) + t
>>> M = S.quotient_module(P**Integer(2), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3
```

We create a submodule, which is not saturated:

```
sage: N = M.span(X^3*P*e0)
sage: N.basis()
[(t^3+2*t^2)*e0 + (t^2+2*t)*e2, (t^2+2*t+1)*e1 + (t+1)*e3]
```

```
>>> from sage.all import *
>>> N = M.span(X**Integer(3)*P*e0)
>>> N.basis()
[(t^3+2*t^2)*e0 + (t^2+2*t)*e2, (t^2+2*t+1)*e1 + (t+1)*e3]
```

and compute its saturation:

```
sage: Nsat = N.saturate()
sage: Nsat.basis()
[t*e0 + e2, (t+1)*e1 + e3]
```

```
>>> from sage.all import *
>>> Nsat = N.saturate()
>>> Nsat.basis()
[t*e0 + e2, (t+1)*e1 + e3]
```

One can check that `Nsat` is the submodule generated by $MP$:

```
sage: Nsat == M.span(P*e0)
True
```

```
>>> from sage.all import *
>>> Nsat == M.span(P*e0)
True
```

**class** sage.modules.ore_module.**ScalarAction**

Bases: Action

Action by scalar multiplication on Ore modules.

sage.modules.ore_module.**normalize_names**(*names*, *rank*)

Return a normalized form of names.

INPUT:

- names – a string, a list of strings or None
- rank – the number of names to normalize

EXAMPLES:

```
sage: from sage.modules.ore_module import normalize_names
```

```
>>> from sage.all import *
>>> from sage.modules.ore_module import normalize_names
```

When names is a string, indices are added:

```
sage: normalize_names('e', 3)
('e0', 'e1', 'e2')
```

```
>>> from sage.all import *
>>> normalize_names('e', Integer(3))
('e0', 'e1', 'e2')
```

When names is a list or a tuple, it remains untouched except that it is always casted to a tuple (in order to be hashable and serve as a key):

```
sage: normalize_names(['u', 'v', 'w'], 3)
('u', 'v', 'w')
```

```
>>> from sage.all import *
>>> normalize_names(['u', 'v', 'w'], Integer(3))
('u', 'v', 'w')
```

Similarly, when names is None, nothing is returned:

```
sage: normalize_names(None, 3)
```

```
>>> from sage.all import *
>>> normalize_names(None, Integer(3))
```

If the number of names is not equal to rank, an error is raised:

```
sage: normalize_names(['u', 'v', 'w'], 2)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: the number of given names does not match the rank of the Ore module
```

```
>>> from sage.all import *
>>> normalize_names(['u', 'v', 'w'], Integer(2))
Traceback (most recent call last):
...
ValueError: the number of given names does not match the rank of the Ore module
```

## 1.2 Elements in Ore modules

AUTHOR:

- Xavier Caruso (2024-10)

**class** sage.modules.ore_module_element.**OreModuleElement**

Bases: FreeModuleElement_generic_dense

A generic element of a Ore module.

**image**(*integral=False*)

Return the image of this element by the pseudomorphism defining the action of the Ore variable on this Ore module.

INPUT:

- `integral` (default: `False`) – a boolean; if `False`, allows for an output with coordinates in the fraction field of the base ring

EXAMPLES:

```
sage: A.<t> = QQ['t']
sage: d = A.derivation()
sage: S.<X> = OrePolynomialRing(A, A.derivation())
sage: M.<v,w> = S.quotient_module(X^2 + t)
sage: v.image()
w
sage: w.image()
-t*v
```

```
>>> from sage.all import *
>>> A = QQ['t']; (t,) = A._first_ngens(1)
>>> d = A.derivation()
>>> S = OrePolynomialRing(A, A.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t, names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> v.image()
w
>>> w.image()
-t*v
```

**is_mutable**()

Always return `False` since elements in Ore modules are all immutable.

EXAMPLES:

```
sage: K.<t> = Frac(QQ['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: M = S.quotient_module(X^2 + t)

sage: v, w = M.basis()
sage: v
(1, 0)
sage: v.is_mutable()
False
sage: v[1] = 1
Traceback (most recent call last):
...
ValueError: vectors in Ore modules are immutable
```

```
>>> from sage.all import *
>>> K = Frac(QQ['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t)

>>> v, w = M.basis()
>>> v
(1, 0)
>>> v.is_mutable()
False
>>> v[Integer(1)] = Integer(1)
Traceback (most recent call last):
...
ValueError: vectors in Ore modules are immutable
```

**vector**()

Return the coordinates vector of this element.

EXAMPLES:

```
sage: K.<t> = Frac(QQ['t'])
sage: S.<X> = OrePolynomialRing(K, K.derivation())
sage: M.<v,w> = S.quotient_module(X^2 + t)
sage: v.vector()
(1, 0)
```

```
>>> from sage.all import *
>>> K = Frac(QQ['t'], names=('t',)); (t,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.derivation(), names=('X',)); (X,) = S._first_
↪ngens(1)
>>> M = S.quotient_module(X**Integer(2) + t, names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> v.vector()
(1, 0)
```

We underline that this vector is not an element of the Ore module; it lives in $K^2$. Compare:

```
sage: v.parent()
Ore module <v, w> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
sage: v.vector().parent()
Vector space of dimension 2 over Fraction Field of Univariate Polynomial Ring␣
↪in t over Rational Field
```

```
>>> from sage.all import *
>>> v.parent()
Ore module <v, w> over Fraction Field of Univariate Polynomial Ring in t over␣
↪Rational Field twisted by d/dt
>>> v.vector().parent()
Vector space of dimension 2 over Fraction Field of Univariate Polynomial Ring␣
↪in t over Rational Field
```

CHAPTER

TWO

# MORPHISMS

## 2.1 Space of morphisms between Ore modules

AUTHOR:

- Xavier Caruso (2024-10)

**class** sage.modules.ore_module_homspace.**OreModule_homspace**(*domain*, *codomain*, *category=None*)

Bases: UniqueRepresentation, HomsetWithBase

Class for hom spaces between Ore modules.

**Element**

alias of *OreModuleMorphism*

**identity**()

Return the identity morphism in this homspace.

EXAMPLES:

```
sage: K.<z> = GF(7^2)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^3 + z*X + 1)
sage: End(M).identity()
Ore module endomorphism of Ore module of rank 3 over Finite Field in z of␣
↪size 7^2 twisted by z |--> z^7
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(2), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) + z*X + Integer(1))
>>> End(M).identity()
Ore module endomorphism of Ore module of rank 3 over Finite Field in z of␣
↪size 7^2 twisted by z |--> z^7
```

**matrix_space**()

Return the matrix space used to represent the morphisms in this homspace.

EXAMPLES:

```
sage: K.<z> = GF(7^2)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
```

(continues on next page)

(continued from previous page)

```
sage: M = S.quotient_module(X^3 + z*X + 1)
sage: End(M).matrix_space()
Full MatrixSpace of 3 by 3 dense matrices over Finite Field in z of size 7^2
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(2), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) + z*X + Integer(1))
>>> End(M).matrix_space()
Full MatrixSpace of 3 by 3 dense matrices over Finite Field in z of size 7^2
```

```
sage: N = S.quotient_module(X^2 + z)
sage: Hom(M, N).matrix_space()
Full MatrixSpace of 3 by 2 dense matrices over Finite Field in z of size 7^2
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(2) + z)
>>> Hom(M, N).matrix_space()
Full MatrixSpace of 3 by 2 dense matrices over Finite Field in z of size 7^2
```

**zero**()

Return the zero morphism in this homspace.

EXAMPLES:

```
sage: K.<z> = GF(7^2)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^3 + z*X + 1)
sage: End(M).zero()
Ore module endomorphism of Ore module of rank 3 over Finite Field in z of␣
↪size 7^2 twisted by z |--> z^7
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(2), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(3) + z*X + Integer(1))
>>> End(M).zero()
Ore module endomorphism of Ore module of rank 3 over Finite Field in z of␣
↪size 7^2 twisted by z |--> z^7
```

## 2.2 Morphisms between Ore modules

Let $R$ be a commutative ring, $\theta : R \to R$ by a ring endomorphism and $\partial : R \to R$ be a $\theta$-derivation. Let also $\mathcal{S} = R[X; \theta, \partial]$ denote the associated Ore polynomial ring.

By definition, a Ore module is a module over $\mathcal{S}$. In SageMath, there are rather represented as modules over $R$ equipped with the map giving the action of the Ore variable $X$. We refer to *sage.modules.ore_module* for more details.

A morphism of Ore modules is a $R$-linear morphism commuting with the Ore action, or equivalenty a $\mathcal{S}$-linear map.

### Construction of morphisms

There are several ways for creating Ore modules morphisms in SageMath. First of all, one can use the method *sage.modules.ore_module.OreModule.hom()*, passing to it the matrix (in the canonical bases) of the morphism we want to build:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<e0,e1> = S.quotient_module(X^2 + X + z)

sage: mat = matrix(2, 2, [z,     3*z^2 + z + 2,
....:                     z + 1,       4*z + 4])
sage: f = M.hom(mat)
sage: f
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,) = S._
↪first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + X + z, names=('e0', 'e1',)); (e0, e1,) = M._
↪first_ngens(2)

>>> mat = matrix(Integer(2), Integer(2), [z,     Integer(3)*z**Integer(2) + z +␣
↪Integer(2),
...                     z + Integer(1),       Integer(4)*z + Integer(4)])
>>> f = M.hom(mat)
>>> f
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
```

Clearly, this method is not optimal: typing all the entries of the defining matrix is long and a potential source of errors.

Instead, one can use a dictionary encoding the values taken by the morphism on a set of generators; the morphism is then automatically prolonged by $\mathcal{S}$-linearity. Actually here, $f$ was just the multiplication by $X^3$ on $M$. We can then redefine it simply as follows:

```
sage: g = M.hom({e0: X^3*e0})
sage: g
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> g = M.hom({e0: X**Integer(3)*e0})
>>> g
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
```

One can then recover the matrix by using the method *sage.modules.ore_module_morphism.OreModuleMorphism.matrix()*:

```
sage: g.matrix()
[            z 3*z^2 + z + 2]
[        z + 1       4*z + 4]
```

```
>>> from sage.all import *
>>> g.matrix()
[            z 3*z^2 + z + 2]
[        z + 1       4*z + 4]
```

Alternatively, one can use the method *sage.modules.ore_module.OreModule.multiplication_map()*:

```
sage: h = M.multiplication_map(X^3)
sage: h
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
sage: g == h
True
```

```
>>> from sage.all import *
>>> h = M.multiplication_map(X**Integer(3))
>>> h
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
>>> g == h
True
```

Of course, this method also accepts values in the base ring:

```
sage: h = M.multiplication_map(2)
sage: h
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
sage: h.matrix()
[2 0]
[0 2]
```

```
>>> from sage.all import *
>>> h = M.multiplication_map(Integer(2))
>>> h
Ore module endomorphism of Ore module <e0, e1> over Finite Field in z of size 5^3␣
↪twisted by z |--> z^5
>>> h.matrix()
[2 0]
[0 2]
```

Be careful that scalar multiplications do not always properly define a morphism of Ore modules:

```
sage: M.multiplication_map(z)
Traceback (most recent call last):
...
ValueError: does not define a morphism of Ore modules
```

```
>>> from sage.all import *
>>> M.multiplication_map(z)
Traceback (most recent call last):
...
ValueError: does not define a morphism of Ore modules
```

SageMath provides methods to compute kernels, cokernels, images and coimages. In order to illustrate this, we will build the sequence

$$0 \to \mathcal{S}/\mathcal{S}P \to \mathcal{S}/\mathcal{S}PQ \to \mathcal{S}/\mathcal{S}Q \to 0$$

and check that it is exact. We first build the Ore modules:

```
sage: P = X^2 + z*X + 1
sage: Q = X^3 + z^2*X^2 + X + z
sage: U = S.quotient_module(P, names='u')
sage: U.inject_variables()
Defining u0, u1
sage: V = S.quotient_module(P*Q, names='v')
sage: V.inject_variables()
Defining v0, v1, v2, v3, v4
sage: W = S.quotient_module(Q, names='w')
sage: W.inject_variables()
Defining w0, w1, w2
```

```
>>> from sage.all import *
>>> P = X**Integer(2) + z*X + Integer(1)
>>> Q = X**Integer(3) + z**Integer(2)*X**Integer(2) + X + z
>>> U = S.quotient_module(P, names='u')
>>> U.inject_variables()
Defining u0, u1
>>> V = S.quotient_module(P*Q, names='v')
>>> V.inject_variables()
Defining v0, v1, v2, v3, v4
>>> W = S.quotient_module(Q, names='w')
>>> W.inject_variables()
Defining w0, w1, w2
```

Next, we build the morphisms:

```
sage: f = U.hom({u0: Q*v0})
sage: g = V.hom({v0: w0})
```

```
>>> from sage.all import *
>>> f = U.hom({u0: Q*v0})
>>> g = V.hom({v0: w0})
```

We can now check that $f$ is injective by computing its kernel:

```
sage: f.kernel()
Ore module of rank 0 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> f.kernel()
Ore module of rank 0 over Finite Field in z of size 5^3 twisted by z |--> z^5
```

We see on the output that it has dimension 0; so it vanishes. Instead of reading the output, one can check programmatically the vanishing of a Ore module using the method *sage.modules.ore_module.OreModule.is_zero()*:

```
sage: f.kernel().is_zero()
True
```

```
>>> from sage.all import *
>>> f.kernel().is_zero()
True
```

Actually, in our use case, one can, more simply, use the method *sage.modules.ore_module_morphism.OreModuleMorphism.is_injective()*:

```
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> f.is_injective()
True
```

Similarly, one checks that $g$ is surjective:

```
sage: g.is_surjective()
True
```

```
>>> from sage.all import *
>>> g.is_surjective()
True
```

or equivalently:

```
sage: g.cokernel().is_zero()
True
```

```
>>> from sage.all import *
>>> g.cokernel().is_zero()
True
```

Now, we need to check that the kernel of $g$ equals the image of $f$. For this, we compute both and compare the results:

```
sage: ker = g.kernel()
sage: im = f.image()
sage: ker == im
True
```

```
>>> from sage.all import *
>>> ker = g.kernel()
>>> im = f.image()
>>> ker == im
True
```

As a sanity check, one can also verity that the composite $g \circ f$ vanishes:

```
sage: h = g * f
sage: h
```

(continues on next page)

(continued from previous page)

```
Ore module morphism:
  From: Ore module <u0, u1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <w0, w1, w2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
sage: h.is_zero()
True
```

```
>>> from sage.all import *
>>> h = g * f
>>> h
Ore module morphism:
  From: Ore module <u0, u1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <w0, w1, w2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
>>> h.is_zero()
True
```

Let us now consider another morphism $f$ and build the canonical isomorphism coim $f \to$ im $f$ that it induces. We start by defining $f$:

```
sage: A = X + z
sage: B = X + z + 1
sage: P = X^2 + X + z
sage: U = S.quotient_module(B*P, names='u')
sage: U.inject_variables()
Defining u0, u1, u2
sage: V = S.quotient_module(P*A, names='v')
sage: V.inject_variables()
Defining v0, v1, v2

sage: f = U.hom({u0: A*v0})
sage: f
Ore module morphism:
  From: Ore module <u0, u1, u2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
  To:   Ore module <v0, v1, v2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
```

```
>>> from sage.all import *
>>> A = X + z
>>> B = X + z + Integer(1)
>>> P = X**Integer(2) + X + z
>>> U = S.quotient_module(B*P, names='u')
>>> U.inject_variables()
Defining u0, u1, u2
>>> V = S.quotient_module(P*A, names='v')
>>> V.inject_variables()
Defining v0, v1, v2

>>> f = U.hom({u0: A*v0})
>>> f
Ore module morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Ore module <u0, u1, u2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
  To:   Ore module <v0, v1, v2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
```

Now we compute the image and the coimage:

```
sage: I = f.image(names='im')
sage: I
Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5

sage: C = f.coimage(names='co')
sage: C
Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> I = f.image(names='im')
>>> I
Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5

>>> C = f.coimage(names='co')
>>> C
Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

We can already check that the image and the coimage have the same rank. We now want to construct the isomorphism between them. For this, we first need to corestrict $f$ to its image. This is achieved via the method *sage.modules.ore_module.OreSubmodule.morphism_corestriction()* of the Ore module:

```
sage: g = I.morphism_corestriction(f)
sage: g
Ore module morphism:
  From: Ore module <u0, u1, u2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> g = I.morphism_corestriction(f)
>>> g
Ore module morphism:
  From: Ore module <u0, u1, u2> over Finite Field in z of size 5^3 twisted by z |-->␣
↪z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

Next, we want to factor $g$ by the coimage. We proceed as follows:

```
sage: h = C.morphism_quotient(g)
sage: h
Ore module morphism:
  From: Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

```
>>> from sage.all import *
>>> h = C.morphism_quotient(g)
>>> h
Ore module morphism:
  From: Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
```

We have found the morphism we were looking for: it is $h$. We can now check that it is an isomorphism:

```
sage: h.is_isomorphism()
True
```

```
>>> from sage.all import *
>>> h.is_isomorphism()
True
```

As a shortcut, we can use explicit conversions as follows:

```
sage: H = Hom(C, I)  # the hom space
sage: h2 = H(f)
sage: h2
Ore module morphism:
  From: Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: h == h2
True
```

```
>>> from sage.all import *
>>> H = Hom(C, I)  # the hom space
>>> h2 = H(f)
>>> h2
Ore module morphism:
  From: Ore module <co0, co1> over Finite Field in z of size 5^3 twisted by z |--> z^5
  To:   Ore module <im0, im1> over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> h == h2
True
```

For endomorphisms, one can compute classical invariants as determinants and characteristic polynomials. To illustrate this, we check on an example that the characteristic polynomial of the multiplication by $X^3$ on the quotient $\mathcal{S}/\mathcal{S}P$ is the reduced norm of $P$:

```
sage: P = X^5 + z*X^4 + z^2*X^2 + z + 1
sage: M = S.quotient_module(P)
sage: f = M.multiplication_map(X^3)
sage: f.charpoly()
x^5 + x^4 + x^3 + x^2 + 1

sage: P.reduced_norm('x')
x^5 + x^4 + x^3 + x^2 + 1
```

```
>>> from sage.all import *
>>> P = X**Integer(5) + z*X**Integer(4) + z**Integer(2)*X**Integer(2) + z + Integer(1)
```

(continues on next page)

(continued from previous page)

```
>>> M = S.quotient_module(P)
>>> f = M.multiplication_map(X**Integer(3))
>>> f.charpoly()
x^5 + x^4 + x^3 + x^2 + 1

>>> P.reduced_norm('x')
x^5 + x^4 + x^3 + x^2 + 1
```

AUTHOR:

- Xavier Caruso (2024-10)

**class** sage.modules.ore_module_morphism.**OreModuleMorphism**(*parent*, *im_gens*, *check=True*)

Bases: Morphism

Generic class for morphism between Ore modules.

**characteristic_polynomial**(*var='x'*)

Return the determinant of this endomorphism.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + (z^2 + 3)*X + z^5
sage: M = S.quotient_module(P)
sage: f = M.multiplication_map(X^3)
sage: f.characteristic_polynomial()
x^3 + x^2 + 2*x + 2
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + (z**Integer(2) + Integer(3))*X +␣
↪z**Integer(5)
>>> M = S.quotient_module(P)
>>> f = M.multiplication_map(X**Integer(3))
>>> f.characteristic_polynomial()
x^3 + x^2 + 2*x + 2
```

We check that the latter is equal to the reduced norm of $P$:

```
sage: P.reduced_norm('x')
x^3 + x^2 + 2*x + 2
```

```
>>> from sage.all import *
>>> P.reduced_norm('x')
x^3 + x^2 + 2*x + 2
```

**charpoly**(*var='x'*)

alias of *characteristic_polynomial()*.

**coimage**(*names=None*)

Return the coimage of this morphism.

INPUT:

- `names` (default: `None`) – the name of the vectors in a basis of the coimage

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = M.hom({m0: n0})
sage: coim = f.coimage()
sage: coim
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: coim.basis()
[m3, m4, m5]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = M.hom({m0: n0})
>>> coim = f.coimage()
>>> coim
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> coim.basis()
[m3, m4, m5]
```

**cokernel**(*remove_torsion=False*, *names=None*)

Return the cokernel of this morphism.

INPUT:

- `remove_torsion` (default: `False`) – a boolean; if `True`, return the cokernel quotiented out by its torsion part
- `names` (default: `None`) – the name of the vectors in a basis of the cokernel

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = N.hom({n0: P*m0})
sage: coker = f.cokernel()
sage: coker
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: coker.basis()
[m3, m4, m5]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = N.hom({n0: P*m0})
>>> coker = f.cokernel()
>>> coker
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> coker.basis()
[m3, m4, m5]
```

**det**()

alias of *determinant()*.

**determinant**()

Return the determinant of this endomorphism.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<m0,m1> = S.quotient_module(X^2 + z)
sage: f = M.multiplication_map(X^3)
sage: f.determinant()
2
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z, names=('m0', 'm1',)); (m0, m1,)␣
↪= M._first_ngens(2)
>>> f = M.multiplication_map(X**Integer(3))
>>> f.determinant()
2
```

If the domain differs from the codomain (even if they have the same rank), an error is raised:

```
sage: N.<n0,n1> = S.quotient_module(X^2 + z^25)
sage: g = M.hom({z*m0: n0})
sage: g.determinant()
Traceback (most recent call last):
...
ValueError: determinants are only defined for endomorphisms
```

```
>>> from sage.all import *
>>> N = S.quotient_module(X**Integer(2) + z**Integer(25), names=('n0', 'n1',
↪)); (n0, n1,) = N._first_ngens(2)
>>> g = M.hom({z*m0: n0})
>>> g.determinant()
Traceback (most recent call last):
...
ValueError: determinants are only defined for endomorphisms
```

**image** (*saturate=False*, *names=None*)

Return the image of this morphism.

INPUT:

- `saturate` (default: `False`) – a boolean; if `True`, return the saturation (in the codomain) of the image (see *sage.modules.ore_module.OreSubmodule.saturate()* for more details)
- `names` (default: `None`) – the name of the vectors in a basis of the image

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = N.hom({n0: P*m0})
sage: im = f.image()
sage: im
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: im.basis()
[m0 + (2*z^2+3*z+1)*m3 + (4*z^2+3*z+3)*m4 + (2*z^2+3*z)*m5,
```

(continues on next page)

(continued from previous page)

```
 m1 + (z+3)*m3 + (z^2+z+4)*m4,
 m2 + (2*z^2+4*z+2)*m4 + (2*z^2+z+1)*m5]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = N.hom({n0: P*m0})
>>> im = f.image()
>>> im
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> im.basis()
[m0 + (2*z^2+3*z+1)*m3 + (4*z^2+3*z+3)*m4 + (2*z^2+3*z)*m5,
 m1 + (z+3)*m3 + (z^2+z+4)*m4,
 m2 + (2*z^2+4*z+2)*m4 + (2*z^2+z+1)*m5]
```

**inverse**()

Return the inverse of this morphism.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^2 + z
sage: M.<e0,e1,e2,e3> = S.quotient_module(P^2)

sage: f = M.multiplication_map(X^3)
sage: g = f.inverse()
sage: (f*g).is_identity()
True
sage: (g*f).is_identity()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(2) + z
>>> M = S.quotient_module(P**Integer(2), names=('e0', 'e1', 'e2', 'e3',));␣
↪(e0, e1, e2, e3,) = M._first_ngens(4)

>>> f = M.multiplication_map(X**Integer(3))
>>> g = f.inverse()
>>> (f*g).is_identity()
```

(continues on next page)

(continued from previous page)

```
True
>>> (g*f).is_identity()
True
```

If the morphism is not invertible, an error is raised:

```
sage: h = M.hom({e0: P*e0})
sage: h.inverse()
Traceback (most recent call last):
...
ValueError: this morphism is not invertible
```

```
>>> from sage.all import *
>>> h = M.hom({e0: P*e0})
>>> h.inverse()
Traceback (most recent call last):
...
ValueError: this morphism is not invertible
```

**is_bijective**()

Return `True` if this morphism is bijective.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: f = M.multiplication_map(X^3)
sage: f.is_bijective()
True

sage: N = S.quotient_module(X^2)
sage: g = N.multiplication_map(X^3)
sage: g.is_bijective()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
>>> f = M.multiplication_map(X**Integer(3))
>>> f.is_bijective()
True

>>> N = S.quotient_module(X**Integer(2))
>>> g = N.multiplication_map(X**Integer(3))
>>> g.is_bijective()
False
```

**is_identity**()

Return `True` if this morphism is the identity.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M.<v,w> = S.quotient_module(X^2 + z)
sage: f = M.hom({v: v})
sage: f.is_identity()
True

sage: f = M.hom({v: 2*v})
sage: f.is_identity()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z, names=('v', 'w',)); (v, w,) = M._
↪first_ngens(2)
>>> f = M.hom({v: v})
>>> f.is_identity()
True

>>> f = M.hom({v: Integer(2)*v})
>>> f.is_identity()
False
```

**is_injective**()

Return `True` if this morphism is injective.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = N.hom({n0: P*m0})
sage: f.is_injective()
True

sage: g = M.hom({m0: n0})
sage: g.is_injective()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
```

(continues on next page)

(continued from previous page)

```
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = N.hom({n0: P*m0})
>>> f.is_injective()
True

>>> g = M.hom({m0: n0})
>>> g.is_injective()
False
```

**is_isomorphism**()

Return `True` if this morphism is an isomorphism.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: f = M.multiplication_map(X^3)
sage: f.is_isomorphism()
True

sage: N = S.quotient_module(X^2)
sage: g = N.multiplication_map(X^3)
sage: g.is_isomorphism()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
>>> f = M.multiplication_map(X**Integer(3))
>>> f.is_isomorphism()
True

>>> N = S.quotient_module(X**Integer(2))
>>> g = N.multiplication_map(X**Integer(3))
>>> g.is_isomorphism()
False
```

**is_surjective**()

Return `True` if this morphism is surjective.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = N.hom({n0: P*m0})
sage: f.is_surjective()
False

sage: g = M.hom({m0: n0})
sage: g.is_surjective()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = N.hom({n0: P*m0})
>>> f.is_surjective()
False

>>> g = M.hom({m0: n0})
>>> g.is_surjective()
True
```

**is_zero**()

Return `True` if this morphism is zero.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2 + 1
sage: M = S.quotient_module(P^2, names='e')
sage: M.inject_variables()
Defining e0, e1, e2, e3, e4, e5

sage: f = M.hom({e0: P*e0})
sage: f.is_zero()
False
```

(continues on next page)

(continued from previous page)

```
sage: (f*f).is_zero()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2) + Integer(1)
>>> M = S.quotient_module(P**Integer(2), names='e')
>>> M.inject_variables()
Defining e0, e1, e2, e3, e4, e5

>>> f = M.hom({e0: P*e0})
>>> f.is_zero()
False
>>> (f*f).is_zero()
True
```

**kernel**(*names=None*)

Return the kernel of this morphism.

INPUT:

- `names` (default: `None`) – the name of the vectors in a basis of the kernel

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: P = X^3 + z*X^2 + z^2
sage: M = S.quotient_module(P^2, names='m')
sage: M.inject_variables()
Defining m0, m1, m2, m3, m4, m5
sage: N = S.quotient_module(P, names='n')
sage: N.inject_variables()
Defining n0, n1, n2

sage: f = M.hom({m0: n0})
sage: ker = f.kernel()
sage: ker
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
sage: ker.basis()
[m0 + (2*z^2+3*z+1)*m3 + (4*z^2+3*z+3)*m4 + (2*z^2+3*z)*m5,
 m1 + (z+3)*m3 + (z^2+z+4)*m4,
 m2 + (2*z^2+4*z+2)*m4 + (2*z^2+z+1)*m5]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> P = X**Integer(3) + z*X**Integer(2) + z**Integer(2)
>>> M = S.quotient_module(P**Integer(2), names='m')
>>> M.inject_variables()
```

(continues on next page)

(continued from previous page)

```
Defining m0, m1, m2, m3, m4, m5
>>> N = S.quotient_module(P, names='n')
>>> N.inject_variables()
Defining n0, n1, n2

>>> f = M.hom({m0: n0})
>>> ker = f.kernel()
>>> ker
Ore module of rank 3 over Finite Field in z of size 5^3 twisted by z |--> z^5
>>> ker.basis()
[m0 + (2*z^2+3*z+1)*m3 + (4*z^2+3*z+3)*m4 + (2*z^2+3*z)*m5,
 m1 + (z+3)*m3 + (z^2+z+4)*m4,
 m2 + (2*z^2+4*z+2)*m4 + (2*z^2+z+1)*m5]
```

**matrix**()

Return the matrix defining this morphism.

EXAMPLES:

```
sage: K.<z> = GF(5^3)
sage: S.<X> = OrePolynomialRing(K, K.frobenius_endomorphism())
sage: M = S.quotient_module(X^2 + z)
sage: f = M.multiplication_map(2)
sage: f.matrix()
[2 0]
[0 2]

sage: g = M.multiplication_map(X^3)
sage: g.matrix()
[            0 3*z^2 + z + 1]
[      2*z + 1             0]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> S = OrePolynomialRing(K, K.frobenius_endomorphism(), names=('X',)); (X,)␣
↪= S._first_ngens(1)
>>> M = S.quotient_module(X**Integer(2) + z)
>>> f = M.multiplication_map(Integer(2))
>>> f.matrix()
[2 0]
[0 2]

>>> g = M.multiplication_map(X**Integer(3))
>>> g.matrix()
[            0 3*z^2 + z + 1]
[      2*z + 1             0]
```

**class** sage.modules.ore_module_morphism.**OreModuleRetraction**

Bases: Map

Conversion (partially defined) map from an ambient module to one of its submodule.

**class** sage.modules.ore_module_morphism.**OreModuleSection**

Bases: Map

Section map of the projection onto a quotient. It is not necessarily compatible with the Ore action.

# PYTHON MODULE INDEX

## m

sage.modules.ore_module, 3
sage.modules.ore_module_element, 56
sage.modules.ore_module_homspace, 59
sage.modules.ore_module_morphism, 60

# INDEX


## A

ambient_module() (*sage.modules.ore_module.OreSubmodule method*), 45
ambient_modules() (*sage.modules.ore_module.OreModule method*), 8
ambient_modules() (*sage.modules.ore_module.OreSubmodule method*), 46

## B

basis() (*sage.modules.ore_module.OreModule method*), 9

## C

characteristic_polynomial() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 68
charpoly() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 68
coimage() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 68
cokernel() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 69
cover() (*sage.modules.ore_module.OreQuotientModule method*), 36
covers() (*sage.modules.ore_module.OreModule method*), 10
covers() (*sage.modules.ore_module.OreQuotientModule method*), 36

## D

det() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 70
determinant() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 70

## E

Element (*sage.modules.ore_module_homspace.OreModule_homspace attribute*), 59
Element (*sage.modules.ore_module.OreModule attribute*), 8

## F

fitting_index() (*sage.modules.ore_module.OreModule method*), 11

## G

gen() (*sage.modules.ore_module.OreModule method*), 13
gens() (*sage.modules.ore_module.OreModule method*), 14

## H

hom() (*sage.modules.ore_module.OreModule method*), 14

## I

identity() (*sage.modules.ore_module_homspace.OreModule_homspace method*), 59
identity_morphism() (*sage.modules.ore_module.OreModule method*), 17
image() (*sage.modules.ore_module_element.OreModuleElement method*), 56
image() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 71
injection_morphism() (*sage.modules.ore_module.OreSubmodule method*), 47
inverse() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 72
is_bijective() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 73
is_identity() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 73
is_injective() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 74
is_isomorphism() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 75
is_mutable() (*sage.modules.ore_module_element.OreModuleElement method*), 56
is_submodule() (*sage.modules.ore_module.OreModule method*), 17
is_surjective() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 75
is_zero() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 76

is_zero() (*sage.modules.ore_module.OreModule method*), 18

## K

kernel() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 77

## M

matrix() (*sage.modules.ore_module_morphism.OreModuleMorphism method*), 78
matrix() (*sage.modules.ore_module.OreModule method*), 19
matrix_space() (*sage.modules.ore_module_homspace.OreModule_homspace method*), 59
module
    sage.modules.ore_module, 3
    sage.modules.ore_module_element, 56
    sage.modules.ore_module_homspace, 59
    sage.modules.ore_module_morphism, 60
module() (*sage.modules.ore_module.OreModule method*), 19
morphism_corestriction() (*sage.modules.ore_module.OreSubmodule method*), 48
morphism_modulo() (*sage.modules.ore_module.OreQuotientModule method*), 38
morphism_quotient() (*sage.modules.ore_module.OreQuotientModule method*), 39
morphism_restriction() (*sage.modules.ore_module.OreSubmodule method*), 49
multiplication_map() (*sage.modules.ore_module.OreModule method*), 20

## N

normalize_names() (*in module sage.modules.ore_module*), 55

## O

ore_ring() (*sage.modules.ore_module.OreModule method*), 22
OreAction (*class in sage.modules.ore_module*), 7
OreModule (*class in sage.modules.ore_module*), 7
OreModule_homspace (*class in sage.modules.ore_module_homspace*), 59
OreModuleElement (*class in sage.modules.ore_module_element*), 56
OreModuleMorphism (*class in sage.modules.ore_module_morphism*), 68
OreModuleRetraction (*class in sage.modules.ore_module_morphism*), 78
OreModuleSection (*class in sage.modules.ore_module_morphism*), 78
OreQuotientModule (*class in sage.modules.ore_module*), 36
OreSubmodule (*class in sage.modules.ore_module*), 45
over_fraction_field() (*sage.modules.ore_module.OreModule method*), 23
over_fraction_field() (*sage.modules.ore_module.OreQuotientModule method*), 40
over_fraction_field() (*sage.modules.ore_module.OreSubmodule method*), 50

## P

projection_morphism() (*sage.modules.ore_module.OreQuotientModule method*), 41
pseudohom() (*sage.modules.ore_module.OreModule method*), 25

## Q

quo() (*sage.modules.ore_module.OreModule method*), 25
quotient() (*sage.modules.ore_module.OreModule method*), 25

## R

random_element() (*sage.modules.ore_module.OreModule method*), 28
relations() (*sage.modules.ore_module.OreQuotientModule method*), 42
rename_basis() (*sage.modules.ore_module.OreModule method*), 29
rename_basis() (*sage.modules.ore_module.OreQuotientModule method*), 43
rename_basis() (*sage.modules.ore_module.OreSubmodule method*), 51

## S

sage.modules.ore_module
    module, 3
sage.modules.ore_module_element
    module, 56
sage.modules.ore_module_homspace
    module, 59
sage.modules.ore_module_morphism
    module, 60
saturate() (*sage.modules.ore_module.OreSubmodule method*), 53
ScalarAction (*class in sage.modules.ore_module*), 55
span() (*sage.modules.ore_module.OreModule method*), 31
submodule() (*sage.modules.ore_module.OreModule method*), 34

## T

twisting_derivation() (*sage.modules.ore_module.OreModule method*), 34
twisting_morphism() (*sage.modules.ore_module.OreModule method*), 35

## V

vector() (*sage.modules.ore_module_element.OreModuleElement method*), 57

## Z

zero() (*sage.modules.ore_module_homspace.OreModule_homspace method*), 60